Artificial Intelligence
Homework 4

Prof. Truong-Son Hy

October 2, 2025

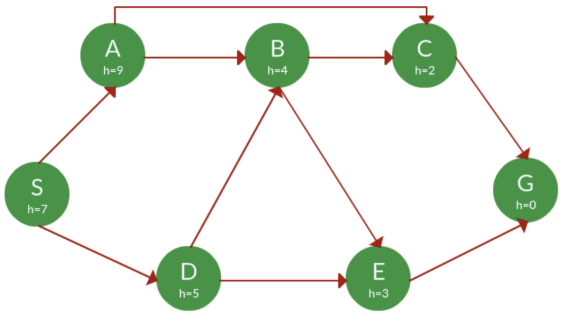
Deadline

October 9, 2025

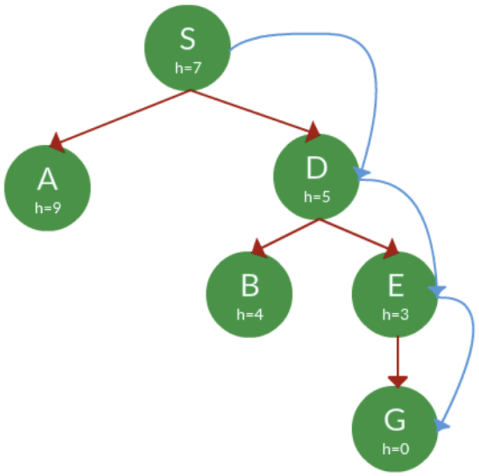
Note: In this assignment, you will need to modify the codes implemented during class. For Greedy Search and A* Tree Search, you can modify the code for **DFS**. For A* Graph Search, you can modify the code for **Dijkstra algorithm**. Remember to submit your codes and report that includes your final results.

Problem 1 (Greedy Search) - 20 points

Graph



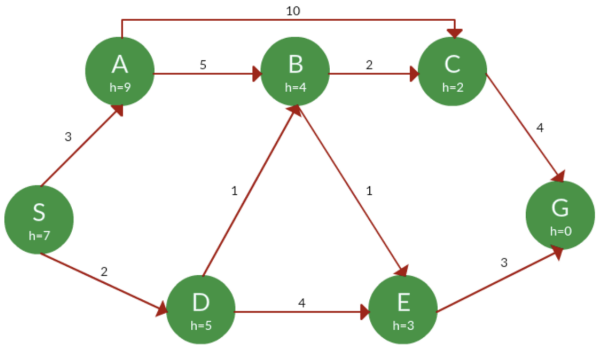
Solution: $S \to D \to E \to G$



Task: You need to implement the Greedy Search by modifying the code during lecture. For the example (above) that we went through during lecture, the Greedy Search must return the correct answer.

Problem 2 (A* Tree Search) - 20 points

Graph

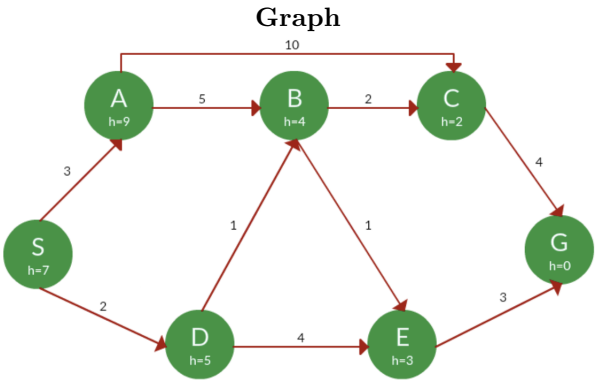


Path: $S \rightarrow D \rightarrow B \rightarrow E \rightarrow G$. **Cost:** 7.

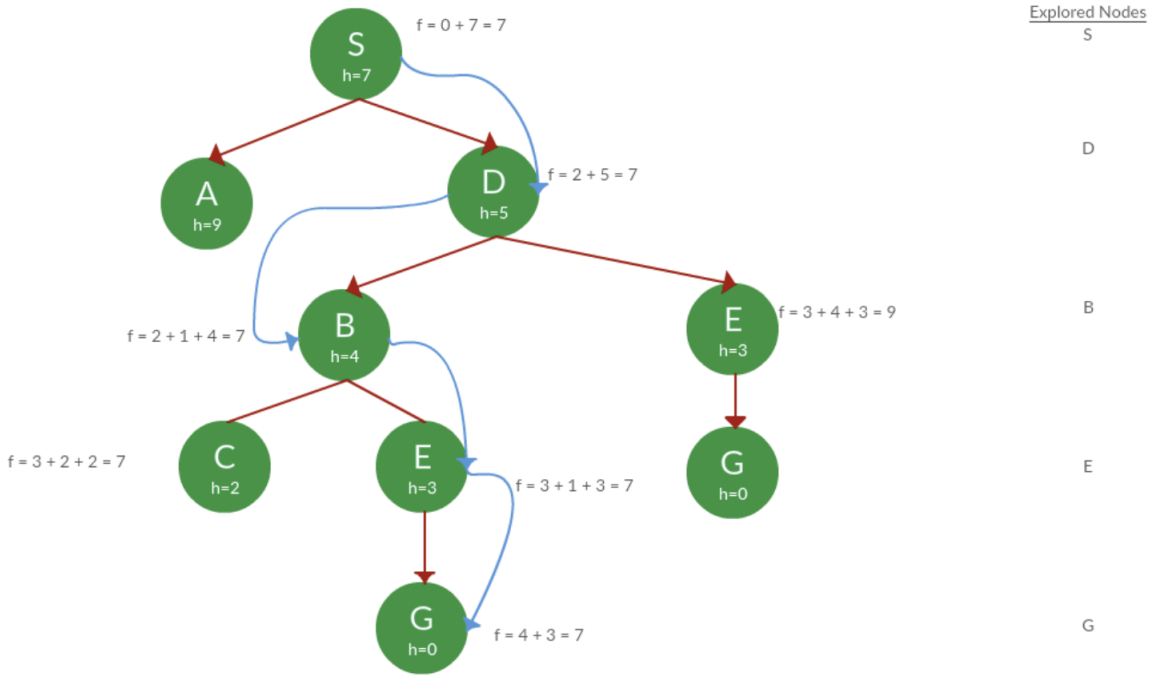
Path	h(x)	g(x)	f(x)
S	7	0	7
S -> A	9	3	12
S -> D ✓	5	2	7
S -> D -> B ✓	4	2 + 1 = 3	7
S -> D -> E	3	2 + 4 = 6	9
S -> D -> B -> C ✓	2	3 + 2 = 5	7
S -> D -> B -> E ✓	3	3 + 1 = 4	7
S -> D -> B -> C -> G	0	5 + 4 = 9	9
S -> D -> B -> E -> G ✓	0	4 + 3 = 7	7

Task: You need to implement the A* Tree Search. For the example (above) that we went through during lecture, the A* Tree Search must return the correct answer.

Problem 3 (A* Graph Search) - 20 points



Path: $S \rightarrow D \rightarrow B \rightarrow E \rightarrow G$. **Cost:** 7.



Task: You need to implement the A* Graph Search. For the example (above) that we went through during lecture, the A* Graph Search must return the correct answer.

Problem 4 (The Knight's Tour Problem) - 20 points

Objective: The goal of this problem is to find a path for a knight on a $N \times N$ chessboard such that the knight visits every square exactly once. In graph theory, this problem is called **Hamiltonian path**.

Problem Description: In chess, the knight moves in an "L" shape: two squares in one direction (horizontal or vertical) and then one square perpendicular, or one square in one direction and two squares perpendicular. Given this unique movement, your task is to write an algorithm that determines a path for the knight to visit every square on the board exactly once, **starting from any given position on the board**.

Note: You can implement this in a language of your choice, but the solution must be efficient and handle the backtracking nature of the problem.

Example 1: If $N = 6$ and the starting cell is $(1, 1)$, we can have this solution (there might be many solutions).
1 28 7 20 3 26
10 21 2 27 8 19
29 36 9 6 25 4
22 11 24 33 18 15
35 30 13 16 5 32
12 23 34 31 14 17

Example 2: If $N = 8$ and the starting cell is $(1, 1)$, we can have this solution (there might be many solutions).
1 4 13 24 27 22 15 18
12 25 2 21 14 17 40 29
3 64 5 26 23 28 19 16
6 11 46 43 20 39 30 41
63 44 7 38 47 42 59 32
10 37 48 45 60 31 56 53
49 62 35 8 51 54 33 58
36 9 50 61 34 57 52 55

Problem 5 (Intelligent Agent) - 20 points

Story. You are given a rectangular grid ("the sea"). Each cell contains either:

- a non-negative integer $w \in \{0, 1, \ldots, 100\}$, or
- a special symbol `S` (start) or `G` (goal).

Integers represent *difficulty* for that sea area. The submarine:

- May move only in the four cardinal directions: Up, Down, Left, Right; **or** North (`N`), South (`S`), West (`W`), East (`E`).
- **Cannot enter islands**: any cell whose integer is 0 is an island and is impassable.
- Must **alternate directions**: Two consecutive moves *cannot* have the same direction (e.g., `E` then `E` is illegal; `E` then `N` is fine).

Cost. Cells `S` and `G` have no difficulty (cost 0). The total path cost is the sum of difficulties on all *entered* cells along the path (excluding `S` and `G`). Your goal is to compute the minimum total difficulty to reach `G` from `S` while satisfying all constraints. If no legal path exists, output `-1`.

Input & Output Format

Input. All input must be read from a file named `submarine.in`. The first line contains an integer T ($T = 10$ in our tests): the number of cases. Each test case has:

- One line with two integers R and C ($1 \leq R, C \leq 100$): rows and columns.
- R lines follow; each line has C tokens. Each token is either:
  - an integer in $[0, 100]$,
  - the character `S`, or
  - the character `G`.

Output. All output must be written to a file named `submarine.out`. For each test case, print a single integer: the minimum total difficulty of a legal path from `S` to `G`; print `-1` if no such path exists.

Constraints & Tests

- **Small test (10 points).** $T = 10$. Each case has $R, C \leq 10$.
- **Large test (10 points).** $T = 10$. Each case has $R, C \leq 100$.

Cell values are integers in $[0, 100]$ where 0 denotes an island (impassable). `S` and `G` have cost 0.

Example

Sample Input (`submarine.in`)

```
1
2 3
S 1 1
1 1 G
```

Sample Output (`submarine.out`)

```
2
```

Explanation. One optimal legal path is:

$$\texttt{S}(1,1) \xrightarrow{\texttt{E}} (1,2)\ (cost = 1) \xrightarrow{\texttt{S}} (2,2)\ (cost = 1) \xrightarrow{\texttt{E}} \texttt{G}(2,3)\ (cost = 0).$$

Total cost $= 1 + 1 = 2$. The direction sequence `E,S,E` has no consecutive identical moves.

What to Implement (Algorithmic task)

Design and implement an algorithm that, for each test case, outputs the required minimum cost under the constraints. Your solution must handle grids up to 100×100 efficiently.

Output Requirements

- Read all input from `submarine.in`, write all answers to `submarine.out`.
- Print one integer per test case (no extra text).
- If unreachable under the rules, print `-1`.

Edge Cases to Consider

- `S` is adjacent to `G`.
- No legal path due to islands or alternating-direction constraint.
- Multiple equal-cost routes (any minimum is acceptable).
- Large grids where a naive search would time out unless you use a proper priority-queue-based shortest path.

Grading Notes

Your code will be run against both the small test and large test. Correctness under the movement and cost rules is required for full credit. Efficiency will be considered for the large cases.